

030 练习详解

1 不等长批次为何必须加权

7行批次均值2，对总平方误差的贡献为14；1行批次均值10，贡献为10。8行总均值为 $(14 + 10)/8 = 3$ 。错误均值为 $(2 + 10)/2 = 6$ 。

在正确规则下每个样本权重均为1/8。在错误规则下，第一批总权重1/2，均分给7个样本，每行权重1/14；最后一个样本独占另一半，权重1/2。因此它相对每个前批样本被放大7倍，相对正确的自身权重被放大4倍。说“最后批次权重翻倍”既不精确，也忽略了两个比较基准。

整体公式是 $\sum_k n_k \bar{L}_k / \sum_k n_k$ 。若各批大小恰好相等，错误写法碰巧一致，不能据此断定实现正确。测试应故意包含不等长批次。本例验证15行分成7、7、1，正是为了让缺陷可见。

2 第二次更新与丢失动量

第一步后 $w = 1.3, b = 0.2$ ，预测1.5与2.8，残差-0.5与-0.2。按 $L = (e_1^2 + e_2^2)/2$ ，梯度为：

$$g_w = (-0.5) \times 1 + (-0.2) \times 2 = -0.9, \quad g_b = -0.5 - 0.2 = -0.7.$$

第一步缓冲是 $v_w = -3, v_b = -2$ 。动量0.5时第二步新缓冲为 $0.5(-3) - 0.9 = -2.4$ 与 $0.5(-2) - 0.7 = -1.7$ 。学习率0.1给出 $w = 1.54, b = 0.37$ ，预测1.91与3.45，残差-0.09与0.45，损失为 $(0.0081 + 0.2025)/2 = 0.1053$ 。

若丢失缓冲，第二步把当前梯度当作初始缓冲，得到 $w = 1.39, b = 0.27$ 。预测1.66与3.05，残差-0.34与0.05，损失 $(0.1156 + 0.0025)/2 = 0.05905$ 。这个特定例子里，丢失动量反而得到更低的下一步损失；它仍不是原定轨迹。恢复正确性由状态递推是否一致决定，不由临时误差谁更低决定。

该例只隔离状态含义，不是在两个更新点上证明动量优劣。test_08还用100组Fraction可精确表示的梯度序列、每组六步，独立核对PyTorch的参数与缓冲。

3 有梯度与被管理的参数不是一回事

计算图根据实际运算追踪依赖。一个普通叶Tensor设置requires_grad=True之后可以接收grad；优化器只更新初始化时传给它的参数对象，两者没有自动的全局同步关系。

普通Tensor属性可修复为`self.w = nn.Parameter(torch.tensor(1.0))`。普通列表中的层应改为`self.layers = nn.ModuleList([nn.Linear(1, 1)])`，并在forward中明确调用相应元素；ModuleList只负责注册和容器行为，不像Sequential那样自动定义串联forward。

修复后检查named_parameters应看到w、layers.0.weight与layers.0.bias，再创建优化器。如果已经创建了优化器，然后才替换或添加参数，旧优化器不自动获得新的参数对象；应按照确定的模型结构重建或显式管理参数组，不能仅看到新的参数名就假定旧优化器已经覆盖它。

此外，登记成功不表示这一轮forward一定用了每个参数。仍需检查缺失grad；None与全零Tensor也不同。本讲每批反向后要求四组注册参数都有有限梯度，这个检查只适用于所有参数每批都参与的当前网络，不适合不加区分地套到有条件分支的模型。

4 形状与错误广播

输入 X 为 $(B, 2)$ ；mean、scale各为 $(2,)$ ；标准化后仍为 $(B, 2)$ 。W1为 $(4, 2)$ 、b1为 $(4,)$ ；计算 $\tilde{X}W_1^T + b_1$ 后为 $(B, 4)$ ，tanh与Dropout均保持shape。W2为 $(1, 4)$ 、b2为 $(1,)$ ，预测为 $(B, 1)$ ；目标也必须是 $(B, 1)$ 。

若目标是 $(B,)$ ，右侧对齐维度时预测的末轴1扩为B，目标在左侧补一个1再扩为B，得到 (B, B) 。矩阵的第i行第j列是第i个预测减第j个目标。原任务只应比较同一行的预测与目标，因此这不是合理的“自动适配”。

参数总数 $8 + 4 + 4 + 1 = 17$ ；标准化统计量另有4个标量，是持久buffer而不是优化器参数。batch_mse先要求shape完全相同，并要求第二轴为1。多目标任务必须另外明确每元素或每样本的损失定义。

5 检查点为什么保存这些状态

本例保存当前模型参数和mean/scale、优化器超参数与动量缓冲、完成轮次、历史、逐轮训练样本顺序、最佳权重/轮次/指标、配置、明确拆分ID、原始输入摘要、CPU全局RNG、训练loader Generator状态及运行版本。

每个项目都对应一个未来依赖：权重和统计量决定前向；动量影响下一更新；RNG决定Dropout和下一轮样本顺序；配置与数据拆分决定执行哪个问题；历史与最佳记录决定候选选择的连续性。逐轮样本顺序是审计证据，不是完整epoch边界继续训练的必需计算状态，但保留它能精确定位差异。

旧grad无需保存，是因为检查点在一整轮结束后，下一批backward之前一定先zero_grad。模型模式无需保存，是因为每轮显式train，验证显式eval并恢复；这不表示state_dict自动含有training标志。评价loader的生成器不保存，是因为本例num_workers=0、样本读取没有随机操作、顺序固定，它产生的迭代器base seed不进入模型数值。

若改为梯度累积中途保存、随机数据增强、多worker预取、混合精度、学习率调度或GPU，以上省略理由就可能失效。只保存“种子”也不等于保存当前随机状态：种子描述序列起点，状态描述当前已消耗到的位置。

6 最佳候选与最新训练状态

本实验事先规定的总预算是12轮，第12轮结束即完成，没有计划中的第13轮；train_until也会拒绝超出配置预算的轮次。一般地，若原计划仍有未完成的训练预算，续训应装最新完整状态；本实验实际在第4轮中断，就装第4轮完整状态，从第5轮继续到第12轮。不能拿第10轮“最佳权重”与第12轮的动量、随机状态拼在一起声称恢复原轨迹。

在本次规定12轮结束后进行最终测试，应装验证选择的第10轮最佳权重与其buffer。评价不需要动量，因为不做更新。使用clone_state或立即序列化避免best与当前参数共享存储；单纯保存model.state_dict()返回的浅映射后继续训练是不够的。[S1][S5]

本代码每次验证严格更小才更新best，故平局保留先出现的轮次。记录best_loss与best_epoch还可检查恢复时是否与历史的最小值一致。不能用测试误差替换这个选择规则而仍声称测试集封存。

7 怎样证明延续了同一轨迹

先运行连续12轮的参照，在第4轮完整结束时保存检查点，但继续运行到第12轮。另开新进程只读第4轮检查点，恢复相同程序、配置、数据、模型、优化器和随机状态，再跑第5—12轮。

至少比较每轮训练ID顺序、各轮指标、最终每个参数和buffer、所有动量缓冲、两个RNG状态、最佳轮次、最佳权重与历史。仅loss相同不充分：不同参数可能给出相同loss；恢复时漏掉状态也可能恰好在某个点看起来接近。

本讲实际新进程结果resume-final-state.json与未中断final-state.json逐字节相同。额外九组“种子×边界”测试进行嵌套状态逐项比较；三种删状态实验产生非零参数差。这仍只说明记录环境与受支持机制的恢复成立，不保证不同PyTorch版本、不同CPU/GPU或不同并行路径逐位相同。[S6]

8 两个错误验证模式

只用no_grad而保留train模式时，Dropout仍抽掩码，评价结果可以随调用改变；若模型有训练模式会更新buffer的层，还可能改变非学习状态。关闭反向图不会把模块自动切到eval。test中的Dropout探针给相同输入64个1两次前向，确实得到不同输出。

只用eval仍会在梯度启用时记录图。故意调用backward和optimizer.step，参数仍改变；eval没有“禁止学习”的权限控制含义。正式evaluate不调用它们，同时使用no_grad，并核对参数、buffer和已有grad不变。

eval也不应被误解成关闭输入已有的requires_grad。029已经区分过新运算是否被记录与现有对象标志，这里沿用同一原则。恢复各子模块的原模式是为了不在验证结束后无意改变下一段调用行为。

9 实验报告示例

故障：第4轮后只恢复模型，漏载训练加载器随机状态。预期不变量是第5轮起每个训练ID的顺序相同；实际顺序首次改变，完整训练后的参数最大差约0.33410590。隔离实验只省略loader_rng，其余模型、动量、CPU全局RNG、数据与配置均按原状态载入。修复为在开始下一轮迭代前set_state到保存的loader_rng；复测全部checkpoint结构相同。

同样的报告可以用于动量和Dropout随机状态，但“最早证据”不同：缺动量时样本顺序不变，第一步更新量已不同；缺全局RNG时样本顺序不变，但训练前向掩码与梯度可能先变化。定位最早差异比只比较最后一个指标更有诊断意义。

结果边界：本实验72行合成数据、12轮、单进程CPU、固定版本、完整epoch边界。测试MSE约0.13905110只是这12行上的误差。文件哈希保护检查固定字节身份，不验证数据科学合理性；单文件替换不提供多文件断电事务。来源索引见sources.md，公开的verification.json记录实际执行范围。